



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Мониторинг аппаратных ресурсов»

Студент ИУ7-74Б
(Группа)

(Подпись, дата)

Сальников М. А.
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Рязанова Н. Ю.
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ОПРЕДЕЛЕНИЯ	5
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	6
ВВЕДЕНИЕ	7
1 Аналитический раздел	8
1.1 Загружаемые модули ядра Linux	8
1.2 Известные характеристики использования процессора	9
1.2.1 Процессорное время	9
1.2.2 Тактовая частота	10
1.2.3 Количество инструкций за такт	11
1.2.4 Прерывания	11
1.3 Анализ известных способов сбора данных об использовании процессора	12
1.3.1 proc	12
1.3.2 sysfs	13
1.3.3 perf_event.h	14
1.4 Формализация задачи	17
2 Конструкторский раздел	18
3 Технологический раздел	19
ЗАКЛЮЧЕНИЕ	20
ПРИЛОЖЕНИЕ А	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21

РЕФЕРАТ

Расчетно-пояснительная записка № с., № рис., № лист., № ист., № прил.

ОПРЕДЕЛЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие термины с соответствующими определениями.

Термин — определение

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие сокращения и обозначения.

CPU — Central Processing Unit

API — Application Programming Interface

IPC — Instructions Per Cycle

IRQ — Interrupt Request

ВВЕДЕНИЕ

Целью данной курсовой работы является разработка загружаемого модуля ядра Linux, предоставляющего пользователю процессу информацию об использовании системой ядер процессора.

Для достижения данной цели необходимо выполнить следующие задачи:

- проанализировать известные характеристики использования процессора;
- проанализировать известные способы сбора данных об использовании процессора;
- спроектировать алгоритмы для достижения поставленной цели;
- спроектировать структуру загружаемого модуля;
- выбрать наиболее подходящий способ взаимодействия с загружаемым модулем;
- выбрать средства реализации загружаемого модуля и реализовать его.

1 Аналитический раздел

В данном разделе представляются формализация задачи, описания загружаемых модулей Linux, известных характеристик использования процессора и оперативной памяти и анализ известных способов сбора данных об использовании системой процессора и оперативной памяти.

1.1 Загружаемые модули ядра Linux

Загружаемый модуль ядра представляет собой скомпилированный объектный код, предназначенный для динамического расширения функциональности ядра операционной системы без необходимости его перекомпиляции. Данный механизм позволяет инкапсулировать новый код в независимые компоненты, которые могут быть присоединены к исполняемому образу ядра во время его работы или отсоединены от него.

Процесс интеграции модуля заключается в вызове его функции инициализации, которая регистрирует предоставляемые модулем услуги в соответствующих подсистемах ядра. После успешной регистрации функция инициализации завершает работу, а модуль становится частью исполняемого кода ядра. Корректное удаление модуля инициирует выполнение функции завершения, задача которой – осуществить deregистрацию услуг и освободить все занятые системные ресурсы для обеспечения целостности ядра.

Важным аспектом архитектуры модулей является их зависимость от интерфейсов, экспортируемых самим ядром: модуль не может использовать функции, не объявленные в его публичном API. Это ограничение обеспечивает стабильность системы, но создает зависимость модуля от конкретной версии ядра. Основное преимущество использования загружаемых модулей заключается в значительном сокращении цикла разработки и отладки низкоуровневого кода, так как процесс итеративного тестирования не требует перезапуска всей системы.

1.2 Известные характеристики использования процессора

Операционная система использует центральный процессор в качестве ключевого вычислительного ресурса для координации работы всех компонентов системы. Для оценки использования процессора может быть применен набор количественных и качественных характеристик.

1.2.1 Процессорное время

Процессорное время характеризует интервал времени, в течение которого процессор выполнял конкретную задачу. Операционная система классифицирует процессорное время, учет которого ведется для каждого процессорного ядра в отдельности. Классы процессорного времени представлены в таблице 1.1.

Таблица 1.1 – Классы процессорного времени

Класс	Описание
Пользовательское время	Время, затраченное на исполнение кода программ в пространстве пользователя
Системное время	Время, затраченное на исполнение кода операционной системы в режиме ядра
Время ожидания ввода-вывода	Время, в течение которого процессор простаивал в состоянии готовности, ожидая завершения операций ввода-вывода от устройств
Время обработки прерываний	Время, затраченное на обслуживание аппаратных прерываний
Время обработки отложенных прерываний	Время, затраченное на обработку нижних половин прерываний
Время ожидания	Время, в течение которого процессорное ядро было полностью свободно

Для оценки относительной загрузки процессора U_{CPU} за выбранный интервал используется показатель, вычисляемый как отношение времени полезной работы к полному времени:

$$U_{CPU} = \left(1 - \frac{I_{end} - I_{begin}}{T_{end} - T_{begin}}\right) \times 100\% \quad (1.1)$$

где:

1. I_{end} и I_{begin} – время ожидания в конце и начале интервала соответственно;
2. T_{end} и T_{begin} – полное время в конце и начале интервала соответственно.

1.2.2 Тактовая частота

Такт ядра процессора – промежуток между двумя импульсами тактового генератора, который синхронизирует выполнение всех операций процессора. Выполнение различных элементарных операций может занимать от долей такта до многих тактов в зависимости от команды и процессора. Тактовая частота является физической характеристикой процессора, определяющей количество синхронизирующих тактов, выполняемых за одну секунду.

Современным процессорам свойственна способность к динамическому изменению тактовой частоты во время работы. Данный механизм служит для решения двух основных задач: повышения энергоэффективности и управления тепловыделением. Снижение частоты в периоды низкой вычислительной нагрузки позволяет значительно уменьшить энергопотребление и нагрев кристалла, а кратковременное повышение частоты сверх номинала позволяет увеличить производительность, если это позволяет тепловой пакет и система питания.

Ответственность за динамическое изменение тактовой частоты распределена между аппаратным обеспечением и программным обеспечением. Непосредственно генерация тактовой частоты осуществляется аппаратным блоком – тактовым генератором, а логика принятия решения о необходимости и величине изменения частоты может реализовываться как аппаратным обеспечением, так и операционной системой.

1.2.3 Количество инструкций за такт

Количество инструкций за такт (IPC) характеризует эффективность использования процессором его вычислительных ресурсов в течение одного такта и может быть вычислено как:

$$IPC = \frac{N_{instructions}}{N_{cycles}} \quad (1.2)$$

где:

1. $N_{instructions}$ – общее количество исполненных процессором инструкций;
2. N_{cycles} – общее количество затраченных процессорных тактов.

Величина IPC не является постоянной, а изменяется динамически в зависимости от множества факторов. Наиболее значимое влияние оказывает микроархитектура процессора: количество и тип исполнительных устройств, размер и ассоциативность кэш-памяти всех уровней, эффективность предвыборки данных и команд. Со стороны программного обеспечения IPC определяется характером вычислительной нагрузки: интенсивностью и локальностью обращений к памяти, соотношением арифметических, целочисленных и операций с плавающей точкой, плотностью ветвлений.

Непосредственно изменить IPC ни операционная система, ни пользовательское приложение не могут, поскольку эта величина является производной от физических возможностей ядра и скомпилированного машинного кода. Однако разработчики могут косвенно влиять на него через оптимизацию программ: улучшение локальности данных, уменьшение количества условных переходов и выравнивание кода, что позволяет более полно загрузить вычислительные ресурсы процессора и, как следствие, повысить IPC.

1.2.4 Прерывания

С точки зрения процессора, обработка прерывания является синхронной операцией, включающей сохранение аппаратного контекста, смену режима работы на привилегированный и переход по векторному адресу. Ключевое влияние на загрузженность процессора оказывают не сами прерывания, а совокупные затраты времени на выполнение кода их обработчиков.

Для количественной оценки вклада прерываний в общую загрузку процессора может использоваться доля процессорного времени, затрачиваемого на исполнение кода всех обработчиков прерываний:

$$T_{interrupts} = N_{interrupts} \cdot \bar{t}_{interrupts} \quad (1.3)$$

где:

1. $N_{interrupts}$ – общее количество прерываний;
2. $\bar{t}_{interrupts}$ – среднее время выполнения обработчика прерывания.

Тогда относительный вклад в загрузку процессора вычисляется как:

$$U_{CPU}^{interrupts} = \frac{T_{interrupts}}{T_{total}} \times 100\% \quad (1.4)$$

где:

1. $T_{interrupts}$ – доля времени, затрачиваемого на исполнение кода всех обработчиков прерываний;
2. T_{total} – полное время.

1.3 Анализ известных способов сбора данных об использовании процессора

1.3.1 proc

`proc` – это виртуальная файловая система, которая используется в качестве интерфейса к структурам ядра операционной системы. Большинство расположенных в ней файлов доступны только для чтения, но некоторые файлы позволяют изменять переменные ядра.

/proc/stat

Данный файл содержит статистику системы. Общие записи включают:

1. строка с меткой **cpu**, содержащая совокупное время, проведенное процессором в различных режимах с момента загрузки системы;
2. строка с меткой **intr**, содержащая количество прерываний, случившихся с момента загрузки системы;
3. строка с меткой **ctxt**, содержащая количество переключений контекста в системе.

/proc/cpuinfo

Данный файл содержит записи информации о процессоре. Общие записи включают:

1. строки с меткой **cpu cores**, содержащие количество физических и логических ядер процессора;
2. строки с меткой **cpu MHz**, содержащие текущую тактовую частоту конкретного ядра процессора.

/proc/interrupts

Данный файл содержит записи количества аппаратных прерываний с момента запуска системы, соответствующих каждой линии IRQ.

/proc/softirq

Данный файл содержит записи количества программных прерываний с момента запуска системы.

1.3.2 sysfs

sysfs – это виртуальная файловая система, файлы которой содержат информацию об устройствах, модулях ядра, других файловых системах и компонентах ядра.

Директория **/sys/devices/system/cpu/cpu*/cpufreq** содержит файлы с информацией о тактовой частоте конкретного ядра процессора:

1. `scaling_cur_freq` – файл, содержащий текущее значение тактовой частоты ядра процессора;
2. `scaling_min_freq` – файл, содержащий минимальное значение тактовой частоты ядра процессора;
3. `scaling_max_freq` – файл, содержащий максимальное значение тактовой частоты ядра процессора;

Из-за отсутствия поддержки управления частотой процессора в ядре или неподходящей аппаратной платформы данная директория может быть недоступна.

Данная файловая система не предоставляет данных об остальных характеристиках в полном объеме, поскольку её архитектура ориентирована на представление иерархии устройств и их параметров, а не агрегированной статистики работы подсистем ядра.

1.3.3 `perf_event.h`

Заголовочный файл `linux/perf_event.h` является частью API ядра Linux и предоставляет программный интерфейс для взаимодействия с подсистемой мониторинга производительности. Основное назначение заключается в предоставлении механизмов для создания, управления и чтения событий производительности из пространства ядра. Событие – это именованный абстрактный ресурс, потребление которого ядром или процессором может быть измерено.

Структура `struct perf_event` предназначена для определения события. Код данной структуры представлен в листинге 1.1.

Листинг 1.1 – Код структуры `struct perf_event`

```
1 struct perf_event {
2     struct list_head event_entry;
3     struct hlist_node hlist_entry;
4     struct perf_event_attr attr;           // Атрибуты события
5     struct perf_event_context *ctx;       // Контекст события
6     enum perf_event_state state;          // Состояние события
7     atomic64_t count;                     // Значение счетчика
8     // ... другие поля
9 };
```

Структура `struct perf_event_attr` предназначена для определения всех параметров события. Код данной структуры представлен в листинге 1.2.

Листинг 1.2 – Код структуры `struct perf_event_attr`

```
1 struct perf_event_attr {
2     __u32 type;                // Тип события (аппаратное,
    программное, tracepoint)
3     __u32 size;                // Размер структуры (для
    обратной совместимости)
4     __u64 config;              // Специфическая конфигурация
    события
5
6     // Битовые поля флагов:
7     __u64 sample_period;       // Период сэмплирования
8     __u64 sample_type;         // Тип данных в сэмплах
9     __u64 read_format;         // Формат данных при чтении
10    __u64 disabled : 1,         // Начинать в выключенном
    состоянии
11        inherit : 1,           // Наследование дочерним
    процессам
12        pinned : 1,            // Закрепление на CPU
13        exclusive : 1,         // Эксклюзивный доступ к PMU
14        exclude_user : 1,      // Исключить пользовательский
    режим
15        exclude_kernel : 1,    // Исключить режим ядра
16        exclude_hv : 1,        // Исключить гипервизор
17        exclude_idle : 1,      // Исключить время простоя
18        // ... другие флаги
19    __u64 wakeup_events;        // Количество событий для wakeup
20    __u32 bp_type;              // Тип точки останова
21    __u64 bp_addr;              // Адрес точки останова
22    __u64 bp_len;               // Длина точки останова
23    // ... другие поля
24 };
```

Перечислимый тип `enum perf_type_id` представляет тип события. Код данного типа представлен в листинге 1.3.

Листинг 1.3 – Код перечислимого типа enum perf_type_id

```
1 enum perf_type_id {
2     PERF_TYPE_HARDWARE          = 0,      // Аппаратные события
3     PERF_TYPE_SOFTWARE          = 1,      // Программные события
4     PERF_TYPE_TRACEPOINT        = 2,      // Точки трассировки
5     PERF_TYPE_HW_CACHE          = 3,      // События кэша
6     PERF_TYPE_RAW                = 4,      // Сырые события PMU
7     PERF_TYPE_BREAKPOINT        = 5,      // Точки останова
8     PERF_TYPE_MAX
9 };
```

Перечислимый тип `enum perf_hw_id` представляет тип аппаратного события. Код данного типа представлен в листинге 1.4.

Листинг 1.4 – Код перечислимого типа enum perf_hw_id

```
1 enum perf_hw_id {
2     PERF_COUNT_HW_CPU_CYCLES      = 0,
3     PERF_COUNT_HW_INSTRUCTIONS    = 1,
4     PERF_COUNT_HW_CACHE_REFERENCES = 2,
5     PERF_COUNT_HW_CACHE_MISSES    = 3,
6     PERF_COUNT_HW_BRANCH_INSTRUCTIONS = 4,
7     PERF_COUNT_HW_BRANCH_MISSES    = 5,
8     PERF_COUNT_HW_BUS_CYCLES      = 6,
9     PERF_COUNT_HW_MAX
10 };
```

Основным системным вызовом для работы с событиями является `perf_event_open()`. Он создаёт файловый дескриптор, через который можно получать измерения производительности. Каждый файловый дескриптор соответствует одному измеряемому событию. При этом события можно группировать для одновременного измерения. Сигнатура данного системного вызова представлена в листинге 1.5

Листинг 1.5 – Сигнатура системного вызова perf_event_open()

```
1 int perf_event_open(struct perf_event_attr *attr, pid_t pid, int
    cpu, int group_fd, unsigned long flags);
```

Интерфейс `perf_event.h` не предоставляет исчерпывающих данных о процессорном времени, тактовой частоте и прерываниях, поскольку его архитектурное назначение заключается в мониторинге низкоуровневых событий производительности на основе аппаратных и программных счётчиков, а не в агрегации системной статистики.

Сравнение способов

Сравнительный анализ способов сбора данных об использовании процессора представлен в таблице 1.2.

Таблица 1.2 – Сравнительный анализ способов сбора данных об использовании процессора

Критерий	proc	sysfs	perf_event.h
Данные о процессорном времени	Да	Нет	Нет
Данные о тактовой частоте	Да	Да	Нет
Данные о количестве инструкций за такт	Нет	Нет	Да
Данные о прерываниях	Да	Нет	Нет

1.4 Формализация задачи

Необходимо реализовать загружаемый модуль ядра Linux, предоставляющий информацию из пространства ядра в пространство пользователя об использовании системой за определенный промежуток времени ядер процессора.

2 Конструкторский раздел

3 Технологический раздел

ЗАКЛЮЧЕНИЕ

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе, содержащая n слайдов